

DESIGN_RULES.md

Reglas de Diseño

Este documento define las reglas que deben respetarse durante todo el desarrollo del proyecto.

Estas reglas tienen prioridad sobre las decisiones de implementación.

1. Una clase, una responsabilidad

Cada clase debe resolver un único problema.

Si una clase comienza a tener responsabilidades múltiples, deberá dividirse.

Ejemplos:

- Shaft representa un eje.
 - Gear representa una rueda dentada.
 - GearMesh representa una restricción cinemática.
 - Renderer dibuja.
 - MechanicalSystem coordina el sistema.
-

2. Separación absoluta entre simulación y renderizado

El motor mecánico nunca dependerá de p5.js.

El Renderer nunca modificará el estado físico del sistema.

Toda modificación del estado mecánico deberá realizarse exclusivamente mediante el núcleo de simulación.

3. MechanicalSystem es el propietario del sistema

Toda creación y eliminación de componentes deberá realizarse mediante MechanicalSystem.

Las demás clases nunca crearán componentes por iniciativa propia.

4. El eje es el elemento fundamental

La unidad básica del simulador es el Shaft.

Los engranajes, poleas y otros componentes se montan sobre un eje.

La velocidad angular pertenece al eje.

Nunca al engranaje.

5. Las restricciones son independientes

GearMesh representa únicamente una relación cinemática.

No dibuja.

No mueve componentes.

No crea objetos.

No conoce el resto del sistema.

6. El código debe ser extensible

Las nuevas funcionalidades deberán añadirse mediante nuevas clases.

Siempre que sea posible se evitará modificar clases existentes.

La complejidad deberá crecer por extensión y no por acumulación de excepciones.

7. No romper compatibilidad

Todo mecanismo construido con una versión estable deberá seguir funcionando después de incorporar nuevas funcionalidades.

Las mejoras nunca deberán romper ejemplos anteriores.

8. La API debe parecer mecánica

La interfaz pública deberá expresar mecanismos, no detalles de programación.

Debe ser posible describir un sistema utilizando conceptos propios de la ingeniería mecánica.

Ejemplo:

```
system.createGear(...)
```

es preferible a

```
new Gear(...)
```

porque expresa la construcción de un mecanismo y mantiene el control dentro del sistema.

9. Mantener el Roadmap

Durante una versión estable no se modifica el orden del ROADMAP.

Las nuevas ideas se registran para versiones futuras.

No se incorporan funcionalidades fuera del plan de desarrollo.

10. Documentar antes de ampliar

Toda modificación importante deberá reflejarse primero en la documentación correspondiente.

La documentación forma parte del proyecto.

No es un elemento secundario.

11. El simulador es un laboratorio

El objetivo del proyecto no es producir una animación.

El objetivo es construir una herramienta para estudiar, comprender, diseñar y experimentar con mecanismos.

Toda decisión de diseño deberá favorecer la capacidad de análisis antes que la apariencia gráfica.

12. La arquitectura es un activo

Una solución rápida que degrade la arquitectura nunca será aceptada.

Cuando exista conflicto entre rapidez de implementación y calidad de diseño, prevalecerá la arquitectura.

El tiempo invertido en una buena arquitectura se recuperará durante el crecimiento del proyecto.

13. Evolución controlada

Cada etapa del desarrollo deberá finalizar con una base estable antes de iniciar la siguiente.

No se construirán nuevas funcionalidades sobre código inestable.

Cada versión deberá ser completamente funcional por sí misma.

Principio rector

La meta del proyecto no es únicamente simular un reloj mecánico.

La meta es desarrollar un motor de simulación mecánica suficientemente sólido como para representar cualquier mecanismo basado en transmisiones, utilizando una arquitectura clara, modular, mantenible y extensible.

Un reloj mecánico constituye el objetivo de validación del proyecto, no su límite.